OffscreenCanvas

Baseline Widely available *

Note: This feature is available in Web Workers.

When using the `<canvas>` element or the Canvas API, rendering, animation, and user interaction usually happen on the main execution thread of a web application. The computation relating to canvas animations and rendering can have a significant impact on application performance.

The **`OffscreenCanvas`** interface provides a canvas that can be rendered off screen, decoupling the DOM and the Canvas API so that the `<canvas>` element is no longer entirely dependent on the DOM. Rendering operations can also be run inside a worker context, allowing you to run some tasks in a separate thread and avoid heavy work on the main thread.

`OffscreenCanvas` is a transferable object.

Constructors

`OffscreenCanvas()`

`OffscreenCanvas` constructor. Creates a new `OffscreenCanvas` object.

Instance properties

`OffscreenCanvas.height`

The height of the offscreen canvas.

`OffscreenCanvas.width`

The width of the offscreen canvas.

Instance methods

`OffscreenCanvas.getContext()`

Returns a drawing context for the offscreen canvas, or `null` if the context identifier is not supported, or the offscreen canvas has already been set to a different context mode.

`OffscreenCanvas.convertToBlob()`

Creates a `Blob` object representing the image contained in the canvas.

`OffscreenCanvas.transferToImageBitmap()`

Creates an `ImageBitmap` object from the most recently rendered image of the `OffscreenCanvas`. See its reference for important notes on managing this `ImageBitmap`.

Events

Inherits events from its parent, `EventTarget`.

Listen to these events using `addEventListener()` or by assigning an event listener to the `oneventname` property of this interface.

`contextlost`

Fired if the browser detects that an `OffscreenCanvasRenderingContext2D` context is lost.

`contextrestored`

Fired if the browser successfully restores an `OffscreenCanvasRenderingContext2D` context.

Examples

Synchronous display of frames produced by an `OffscreenCanvas`

One way to use the `OffscreenCanvas` API is to use a rendering context that has been obtained from an `OffscreenCanvas` object to generate new frames. Once a new frame has finished rendering in this context, the `transferToImageBitmap()` method can be called to save the most recent rendered image. This method returns an `ImageBitmap` object, which can be used in a variety of Web APIs and also in a second canvas without creating a transfer copy.

To display the `ImageBitmap`, you can use an `ImageBitmapRenderingContext` context, which can be created by calling `canvas.getContext("bitmaprenderer")` on a (visible) canvas element. This context only provides functionality to replace the canvas's contents with the given `ImageBitmap`. A call to `ImageBitmapRenderingContext.transferFromImageBitmap()` with the previously rendered and saved `ImageBitmap` from the OffscreenCanvas, will display the `ImageBitmap` on the canvas and transfer its ownership to the canvas. A single `OffscreenCanvas` may transfer frames into an arbitrary number of other `ImageBitmapRenderingContext` objects.

Given these two `<canvas>` elements

HTML

```
<canvas id="one"></canvas> <canvas id="two"></canvas>
```

the following code will provide the rendering using `OffscreenCanvas` as described above.

JS

```
const one = document.getElementById("one").getContext("bitmaprenderer");
const two = document.getElementById("two").getContext("bitmaprenderer");

const offscreen = new OffscreenCanvas(256, 256);
const gl = offscreen.getContext("webgl");

// Perform some drawing for the first canvas using the gl context
const bitmapOne = offscreen.transferToImageBitmap();
one.transferFromImageBitmap(bitmapOne);

// Perform some more drawing for the second canvas
const bitmapTwo = offscreen.transferToImageBitmap();
two.transferFromImageBitmap(bitmapTwo);
```

Asynchronous display of frames produced by an `OffscreenCanvas`

Another way to use the `OffscreenCanvas` API, is to call `transferControlToOffscreen()` on a `<canvas>` element, either on a worker or the main thread, which will return an `OffscreenCanvas` object from an `HTMLCanvasElement` object from the main thread. Calling `getContext()` will then obtain a rendering context from that `OffscreenCanvas`.

The `main.js` script (main thread) may look like this:

JS

```
const htmlCanvas = document.getElementById("canvas");
const offscreen = htmlCanvas.transferControlToOffscreen();

const worker = new Worker("offscreen-canvas.js");
worker.postMessage({ canvas: offscreen }, [offscreen]);
```

While the `offscreen-canvas.js` script (worker thread) can look like this:

JS

```
onmessage = (evt) => {
  const canvas = evt.data.canvas;
```

```
  const gl = canvas.getContext("webgl");
  // Perform some drawing using the gl context
};
```

It's also possible to use `requestAnimationFrame()` in workers:

JS

```
onmessage = (evt) => {
  const canvas = evt.data.canvas;
  const gl = canvas.getContext("webgl");

  function render(time) {
    // Perform some drawing using the gl context
    requestAnimationFrame(render);
  }
  requestAnimationFrame(render);
};
```

For a full example, see the OffscreenCanvas example source on GitHub or run the OffscreenCanvas example live.

Specifications

Specification
HTML # the-offscreencanvas-interface

Browser compatibility

Report problems with this compatibility data • View data on GitHub

	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Internet	WebView Android	WebView on iOS
`OffscreenCanvas`	✓ 69	✓ 79	✓ 105	✓ 56	✓ 16.4	✓ 69	✓ 105	✓ 48	✓ 16.4	✓ 10	✓ 69	✓ 16.4
`OffscreenCanvas()` constructor	✓ 69	✓ 79	✓ 105	✓ 56	✓ 16.4	✓ 69	✓ 105	✓ 48	✓ 16.4	✓ 10	✓ 69	✓ 16.4

`contextlost` `event`	✓ 99	✓ 99	✓ 125	✓ 85	⊗ No	✓ 99	✓ 125	✓ 68	⊗ No	✓ 18	✓ 99	⊗ No
`contextrestored` `event`	✓ 99	✓ 99	✓ 125	✓ 85	⊗ No	✓ 99	✓ 125	✓ 68	⊗ No	✓ 18	✓ 99	⊗ No
`convertToBlob`	✓ 69	✓ 79	✓ 105 ···	✓ 56	✓ 16.4	✓ 69	✓ 105 ···	✓ 48	✓ 16.4	✓ 10	✓ 69	✓ 16.4
`option.type` parameter supports `"image/webp"`	✓ 69	✓ 79	✓ 105	✓ 56	⊗ No	✓ 69	✓ 105	✓ 48	⊗ No	✓ 10	✓ 69	⊗ No
`getContext`	✓ 69	✓ 79	✓ 105	✓ 56	✓ 16.4	✓ 69	✓ 105	✓ 48	✓ 16.4	✓ 10	✓ 69	✓ 16.4
`2d` context	✓ 69	✓ 79	✓ 105	✓ 56	✓ 16.4	✓ 69	✓ 105	✓ 48	✓ 16.4	✓ 10	✓ 69	✓ 16.4
`bitmaprenderer` context	✓ 76	✓ 79	✓ 105	✓ 63	✓ 16.4	✓ 76	✓ 105	✓ 54	✓ 16.4	✓ 12	✓ 76	✓ 16.4
`webgl2` context	✓ 69	✓ 79	✓ 105	✓ 56	✓ 17	✓ 69	✓ 105	✓ 48	✓ 17	✓ 10	✓ 69	✓ 17
`webgl` context	✓ 69	✓ 79	✓ 105	✓ 56	✓ 17	✓ 69	✓ 105	✓ 48	✓ 17	✓ 10	✓ 69	✓ 17
`webgpu` context	⊘ 113	⊘ 113	⊘ 141	⊘ 99	✓ 26	✓ 121	⊗ No	✓ 81	✓ 26	✓ 25	✓ 121	✓ 26
`height`	✓ 69	✓ 79	✓ 105	✓ 56	✓ 16.4	✓ 69	✓ 105	✓ 48	✓ 16.4	✓ 10	✓ 69	✓ 16.4
`transferToImageBitmap`	✓ 69	✓ 79	✓ 105	✓ 56	✓ 16.4	✓ 69	✓ 105	✓ 48	✓ 16.4	✓ 10	✓ 69	✓ 16.4
`width`	✓ 69	✓ 79	✓ 105	✓ 56	✓ 16.4	✓ 69	✓ 105	✓ 48	✓ 16.4	✓ 10	✓ 69	✓ 16.4

✓ Full support ⊘ Partial support ⊗ No support Uses a non-standard name.

··· Has more compatibility info.

See also

- `CanvasRenderingContext2D`
- `OffscreenCanvasRenderingContext2D`
- `ImageBitmap`
- `ImageBitmapRenderingContext`
- `HTMLCanvasElement.transferControlToOffscreen()`
- `requestAnimationFrame()`

- WebGL Off the Main Thread – Mozilla Hacks (2016)

mdn_

Your blueprint for a better internet.